Test Suite (Step-by-Step using Swagger)

1. Start Application

- Import project into Eclipse IDE
- Navigate to main class:
  `MeetingRoomBookingServiceApplication.java`
  Right-click → **Run As → Spring Boot App**
- `Wait until you see:
  `Started Application`

2. Open Swagger

http://localhost:8081/swagger-ui/index.html

STEP 1: CREATE ROOMS

Test 1.1 – Create Room

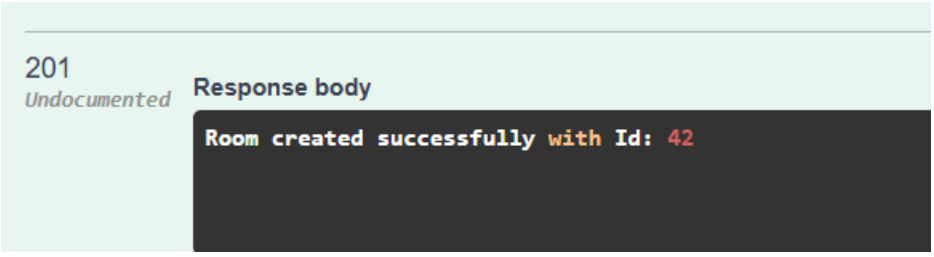
End Point:

POST /rooms/create:

Request:

```
{
   "name": "Room 1",
   "capacity": 10,
    "floor": 1,
    "amenities":[ "WiFi,TV"]
}
```

Expected:



Test 1.2 – Duplicate Room

Same request again

400
Undocumented

Error: response status is 400

Response body

```
{
  "message": "Room name already exists",
  "status": 400,
  "timestamp": "2026-04-21T21:19:26.6711415"
}
```

Test 1.3 – Invalid Capacity

Request:

```
{
  "name": "Room B",
  "capacity": 0,
  "floor": 1,
  "amenities": "AC"
}
```

Expected:

400
Undocumented

Error: response status is 400

Response body

```
{
  "message": "Capacity must be >= 1",
  "status": 400,
  "timestamp": "2026-04-21T21:22:43.2638304"
}
```

STEP 1.4: GET ROOMS (Fetch & Filter)

End Point:

GET /rooms/getRooms

Request URL

```
http://localhost:8081/rooms/getRooms
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Room A",
    "capacity": 10,
    "floor": 2,
    "amenities": [
      "wifi"
    ]
  },
  {
    "id": 2,
    "name": "Room B",
    "capacity": 6,
    "floor": 2,
    "amenities": [
      "AC"
    ]
  },
```


Test 1.4.2 – Filter by Minimum Capacity

GET /rooms?minCapacity=5

Expected

- **Only rooms with capacity ≥ 5**

Request URL

```
http://localhost:8081/rooms/getRooms?minCapacity=5
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Room A",
    "capacity": 10,
    "floor": 2,
    "amenities": [
      "wifi"
    ]
  },
  {
    "id": 2,
    "name": "Room B",
    "capacity": 6,
    "floor": 2,
    "amenities": [
      "AC"
    ]
  },
  {
    "id": 3,
    "name": "Room C",
    "capacity": 6,
    "floor": 2,
    "amenities": [
      "AC"
    ]
```

Test 1.4.3 – Filter by Amenity

GET /rooms?amenity=WiFi

Expected

- **Only rooms containing WiFi**

```
http://localhost:8081/rooms/getRooms?amenity=wifi
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Room A",
    "capacity": 10,
    "floor": 2,
    "amenities": [
      "wifi"
    ]
  }
]
```


Test 1.4.4 – Filter by Capacity + Amenity

GET /rooms?minCapacity=5&amenity=WiFi

```
http://localhost:8081/rooms/getRooms?minCapacity=5&amenity=wifi
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Room A",
    "capacity": 10,
    "floor": 2,
    "amenities": [
      "wifi"
    ]
  }
]
```

Test 1.4.5 – No Matching Rooms

GET /rooms?minCapacity=100

```
http://localhost:8081/rooms/getRooms?minCapacity=100
```

Server response

Code	Details
200	Response body

```
[]
```

STEP 2: CREATE BOOKINGS

Test 2.1 – Booking Success

Endpoint:

POST /bookings

Header

Idempotency-Key: key-1

Request:

```
{
  "roomId": 1,
  "title": "Team Meeting",
  "organizerEmail": "user1@gmail.com",
  "startTime": "2026-04-22T10:00:00",
  "endTime": "2026-04-22T11:00:00"
}
```

Response:

Request URL

```
http://localhost:8081/bookings
```

Server response

Code	Details
201 *Undocumented*	Response body

```
Booking created successfully with Id: 74 and status: confirmed
```

Test 2.2 – Idempotency (Same Key)

dempotency-Key: key-1

Expected

- **Same response**
- **No duplicate booking**

Test 2.3 – Overlapping Booking

Request:

```
{
 "roomId": 1,
 "title": "Overlap",
 "organizerEmail": "user2@gmail.com",
 "startTime": "2026-04-22T10:30:00",
 "endTime": "2026-04-22T11:30:00"
}
```

Response:

Request URL

```
http://localhost:8081/bookings
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400

Response body

```
{
  "message": "Booking overlaps with existing booking",
  "status": 400,
  "timestamp": "2026-04-21T21:56:00.0317763"
}
```

Test 2.4 – Weekend Booking

Request:

```
{
 "roomId": 1,
 "title": "Weekend",
 "organizerEmail": "user@gmail.com",
```

"startTime": "2026-04-25T10:00:00",

"endTime": "2026-04-25T11:00:00"

}

Response:

Request URL

http://localhost:8081/bookings

Server response

Code	Details
400 *Undocumented*	Error: response status is 400 Response body


```
{
  "message": "Bookings allowed only Mon-Fri",
  "status": 400,
  "timestamp": "2026-04-21T21:57:12.6801862"
}
```


Test 2.5 – Invalid Duration

Request:

{

"roomId": 1,

"title": "Short",

"organizerEmail": "user@gmail.com",

"startTime": "2026-04-22T10:00:00",

"endTime": "2026-04-22T10:10:00"

}

Response:

http://localhost:8081/bookings

Server response

Code	Details
400 *Undocumented*	Error: response status is 400 Response body


```
{
  "message": "Duration must be between 15 mins and 4 hours",
  "status": 400,
  "timestamp": "2026-04-21T21:58:20.6415397"
}
```

Test 2.6 – Invalid Time Order

Request:

```
{
 "roomId": 1,
 "title": "Invalid Time",
 "organizerEmail": "user@gmail.com",
 "startTime": "2026-04-22T11:00:00",
 "endTime": "2026-04-22T10:00:00"
}
```

Response:

Request URL

```
http://localhost:8081/bookings
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400 Response body `{ "message": "startTime must be before endTime", "status": 400, "timestamp": "2026-04-21T21:59:33.3933264" }`

Test 2.7 – Outside Working Hours

Request:

```
{
 "roomId": 1,
 "title": "Early Meeting",
 "organizerEmail": "user@gmail.com",
 "startTime": "2026-04-22T07:00:00",
 "endTime": "2026-04-22T08:00:00"
}
```

Response:

Request URL

```
http://localhost:8081/bookings
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400 Response body

```
{
  "message": "Allowed between 08:00-20:00",
  "status": 400,
  "timestamp": "2026-04-21T22:01:17.8677386"
}
```

STEP 3: CANCEL BOOKING

Test 3.1 – Cancel Too Late

Endpoint

POST /bookings/{id}/cancel

```
http://localhost:8081/bookings/41/cancel
```

Server response

Code	Details
400 *Undocumented*	Error: response status is 400 Response body

```
{
  "message": "Cannot cancel within 1 hour of start time",
  "status": 400,
  "timestamp": "2026-04-21T22:04:48.5442171"
}
```

Test 3.2 – Cancel Success

Booking >1 hour away

```
http://localhost:8081/bookings/40/cancel
```

Server response

Code	Details
201 *Undocumented*	Response body

```
Booking canceled successfully
```

STEP 4: GET BOOKINGS

Test 4.1 – Pagination

```
http://localhost:8081/bookings/getBookings?roomId=1&limit=10&offset=0
```

Server response

Code	Details
200	Response body


```
{
  "items": [
    {
      "roomId": 1,
      "title": "Team Meeting",
      "organizerEmail": "abc@gmail.com",
      "startTime": "2026-04-21T10:00:00",
      "endTime": "2026-04-21T11:00:00"
    },
    {
      "roomId": 1,
      "title": "Invalid Cancel Test",
      "organizerEmail": "test@gmail.com",
      "startTime": "2026-04-22T10:30:00",
      "endTime": "2026-04-22T11:00:00"
    },
    {
      "roomId": 1,
      "title": "Team Meeting",
      "organizerEmail": "user1@gmail.com",
      "startTime": "2026-04-22T10:00:00",
      "endTime": "2026-04-22T11:00:00"
    }
  ],
  "total": 3,
  "limit": 10,
  "offset": 0
```


STEP 5: UTILIZATION REPORT

Test 5.1 – Normal Case

http://localhost:8081/bookings/reports/room-utilization?from=2026-05-01T08%3A00%3A00&to=2026-05-01T20%3A00%3A00

Request URL

```
http://localhost:8081/bookings/reports/room-utilization?from=2026-04-22T08%3A00%3A00&to=2026-04-22T20%3A00%3A00
```

Server response

Code	Details
200	Response body


```
[
  {
    "roomId": 1,
    "roomName": "Room A",
    "totalBookingHours": 1,
    "utilizationPercent": 0.08333333333333333
  },
```

Test 5.2 – No Bookings in Range

GET /reports/room-utilization?from=2026-05-01T08:00:00&to=2026-05-01T20:00:00

```
http://localhost:8081/bookings/reports/room-utilization?from=2026-05-02T20%3A00%3A00&to=2026-05-01T08%3A00%3A00
```

Server response

Code	Details

400
Undocumented
	Error: response status is 400
	Response body
	`{ "message": "from date must be before to date", "status": 400, "timestamp": "2026-04-21T22:18:43.9920468" }`
	Response headers
	`connection: close`
	`content-type: application/json`
	`date: Tue,21 Apr 2026 16:48:44 GMT`
	`transfer-encoding: chunked`

Test 6.1 – Same Request Parallel

1. **Open 2 Swagger tabs**
2. **Use same request:**

   Idempotency-Key: parallel-1
3. **Click Execute simultaneously**

   Expected

- **Only 1 booking created**
- **Other request returns same result**
- **No duplicates**